

IES ALBARREGAS

DOCUMENTACIÓN DEL PROYECTO FIN DE GRADO

TUTOR:
JUAN SÁNCHEZ

2ºASIR



ÍNDICE

1. Introducción.....	4
Alcance de la infraestructura.....	4
2. Descripción general de la arquitectura.....	5
Visión global de la infraestructura.....	5
Componentes principales y su función.....	5
Balanceador de carga (Nginx).....	5
Servidores web — Backend (Apache + PHP).....	5
Servidor NFS.....	5
Servidor de base de datos (MySQL 8).....	5
Windows Server (DNS + Active Directory + DHCP).....	6
3. Requisitos del sistema.....	8
3.1 Requisitos hardware.....	8
3.2 Requisitos Software.....	9
Sistema operativo.....	9
Servidores web (Backends).....	9
Servidor NFS.....	9
Balanceador de carga.....	9
Motor de base de datos.....	9
Herramientas de gestión y automatización.....	9
4. Diseño de red.....	10
Topología de red.....	10
Direccionamiento IP.....	10
Puertos y protocolos utilizados.....	10
Esquema de seguridad perimetral.....	11
Principio de defensa en profundidad.....	11
5. Servidores web (Backends).....	13
Función dentro de la infraestructura.....	13
Configuración básica.....	13
Configuración de Apache.....	13
Montaje NFS.....	13
Virtual hosts / sitios web.....	13
6. Balanceador de carga.....	14
Función del balanceador.....	14
Tipo de balanceo.....	14
Configuración del bloque upstream.....	14
7. Base de datos.....	15
Tipo de base de datos.....	15
Arquitectura.....	15

Tablas del esquema clínico.....	15
Usuarios y permisos.....	16
Conexión segura con el servidor web.....	16
8. Dominio Windows Server.....	17
Estructura de unidades organizativas.....	17
Grupos de seguridad.....	17
Servicios DNS y DHCP.....	17
9. Router / NAT — Interconexión entre redes.....	18
Función dentro de la infraestructura.....	18
Direccionamiento.....	18
Ruta estática en el balanceador.....	18
IP Forwarding en el router.....	18
Flujo de comunicación: Balanceador a Windows Server.....	19
10. Sistema de respaldo (Backups).....	21
Objetivos del sistema de copias de seguridad.....	21
Elementos respaldados.....	21
Base de datos.....	21
Código de la aplicación (NFS).....	21
Configuraciones de los servidores.....	22
Procedimiento de restauración.....	22
Restauración de base de datos.....	22
Restauración del código (NFS).....	22
11. Seguridad.....	23
Gestión de usuarios y accesos.....	23
Principio de mínimo privilegio.....	23
Políticas de contraseñas.....	23
Seguridad iptables (SGBD).....	23
Actualizaciones y parches.....	23
Protección frente a ataques comunes.....	24
Buenas prácticas de seguridad.....	24
12. Monitorización y mantenimiento (Script Python).....	25
Herramientas de monitorización.....	25
Parámetros supervisados.....	25
Scripts de monitorización.....	25
Comandos de monitorización manual.....	26
13. Procedimientos operativos.....	28
Arranque y parada de servicios.....	28
Orden de arranque correcto.....	28
6. Orden de parada correcto.....	28
Alta/baja de un servidor backend.....	28
Dar de alta un nuevo backend.....	28
Dar de baja un backend.....	29
Actualización del sistema.....	29
14. Resolución de problemas.....	31

Problemas habituales.....	31
Diagnóstico básico.....	31
Metodología de resolución de problemas.....	31
Comprobaciones recomendadas.....	31
Checklist de verificación de los servidores.....	31
Logs relevantes.....	32
15. Aplicación web — /app (práctica marzo–junio).....	33
Controladores.....	33
Modelos.....	34
Vistas por rol.....	34
Tecnologías usadas en /app.....	34
16. Documentación adicional.....	37
Credenciales de prueba.....	37
Aplicación web.....	37
Servidor FTP.....	37
MySQL.....	37
Tecnologías usadas para elaborar el proyecto y documento.....	38
Virtualización e infraestructura.....	38
Tecnologías empleadas.....	38
Lenguajes.....	38
Diseño del documento.....	38

1. Introducción

Este documento presenta la documentación técnica completa de la infraestructura de red y el sistema de gestión integral desarrollados para la Clínica General. La solución cubre desde los servicios de red hasta la aplicación web de gestión clínica, pasando por la administración de sistemas, la base de datos y la seguridad.

Alcance de la infraestructura

La infraestructura descrita en este documento abarca los siguientes componentes:

- Un balanceador de carga Nginx con ip_hash para distribución de tráfico con persistencia de sesión entre los dos servidores web backend.
- Dos servidores backend con Apache y PHP que montan el código de la aplicación desde un servidor NFS compartido.
- Un servidor de base de datos MySQL 8 con 19 tablas para la gestión clínica completa.
- Un servidor NFS que centraliza el código de la aplicación web para ambos backends.
- Un servidor FTP con vsftpd para el intercambio de archivos entre los departamentos de la clínica.
- Un Windows Server con Active Directory, DNS y DHCP para la gestión del dominio clinicageneral.local.
- Segmentación de red en cuatro capas: LAN Datos, LAN Interna, LAN2 y LAN1.

2. Descripción general de la arquitectura

Visión global de la infraestructura

La infraestructura implementa un modelo de cuatro capas de red con separación de funciones para proporcionar mayor seguridad y disponibilidad. Está compuesta por siete máquinas virtuales Ubuntu 22.04 desplegadas con Vagrant y VirtualBox, interconectadas a través de redes privadas con rangos IP diferenciados por función.

Componentes principales y su función

Balanceador de carga (Nginx)

Función: Recibe todas las peticiones HTTP y las distribuye entre los servidores backend utilizando el algoritmo ip_hash, que garantiza que las sesiones PHP de un mismo cliente siempre lleguen al mismo backend.

Características clave del balanceador:

- Distribución de carga con ip_hash para persistencia de sesión.
- Ruta estática hacia la red de Windows Server (10.0.60.0/23) configurada vía netplan.
- Aislamiento completo de la red de datos (no se accede directamente a la base de datos).

Servidores web — Backend (Apache + PHP)

Función: Procesan las peticiones HTTP y ejecutan el código PHP de la aplicación clínica. Actúan como capa intermedia entre el balanceador y la base de datos.

Características clave:

- Apache 2 con mod_rewrite habilitado y VirtualHost configurado.
- El código de la aplicación se monta desde el servidor NFS (10.0.30.40:/srv/app) vía fstab.
- Doble interfaz de red: LAN Interna (10.0.30.x) y LAN Datos (10.0.20.x).

Servidor NFS

Función: Centraliza el código de la aplicación web para que ambos backends sirvan exactamente el mismo contenido sin necesidad de sincronización manual.

Características clave:

- Exporta /srv/app a toda la red 10.0.30.0/23 con permisos rw, sync, no_subtree_check.
- Elimina la necesidad de desplegar el código en cada backend por separado.

Servidor de base de datos (MySQL 8)

Función: Almacenamiento persistente de todos los datos clínicos. Completamente aislado en la LAN Datos, accesible únicamente desde los backends.

Características clave:

- Configurado para aceptar conexiones únicamente desde los backends (10.0.20.20 y 10.0.20.30).

- 19 tablas que cubren pacientes, citas, historiales, recetas, laboratorio, facturación y control de acceso.
- Contraseñas almacenadas con SHA2(password, 256).

Windows Server (DNS + Active Directory + DHCP)

Función: Gestión centralizada del dominio clinicageneral.local, resolución de nombres, asignación de IPs y control de acceso mediante Active Directory.

Características clave:

- Dominio clinicageneral.local con 8 unidades organizativas y 9 grupos de seguridad.
- DNS con zonas directa e inversa para toda la infraestructura.
- DHCP para la asignación automática de IPs a los equipos de la clínica.

3. Requisitos del sistema

Requisitos hardware

La infraestructura requiere los siguientes recursos para alojar siete máquinas virtuales de forma simultánea:

Componente	CPU	RAM (MB)	Disco (GB)	Interfaces de red
SGBD (MySQL)	1	1024	15	1 (LAN Datos)
Backend 1 (Apache+PHP)	1	512	10	2 (LAN Interna + LAN Datos)
Backend 2 (Apache+PHP)	1	512	10	2 (LAN Interna + LAN Datos)
NFS	1	512	10	1 (LAN Interna)
Balanceador (Nginx)	1	512	10	2 (LAN2 + LAN Interna)
Router/NAT	1	512	10	2 (LAN2 + LAN1)
FTP (vsftpd)	1	512	10	1 (LAN1)
TOTAL	7	4096	75	11

Requisitos mínimos de la máquina física:

- Procesador: CPU con 8 núcleos físicos (mínimo 4).
- RAM: 16 GB (4 GB para las VMs + margen para el sistema operativo host).
- Disco: 100 GB libres (preferiblemente SSD).
- BIOS/UEFI: Virtualización AMD-V o Intel VT-x activada.

Requisitos Software

Sistema operativo

Todas las máquinas virtuales Linux: Ubuntu Server 22.04 LTS (Jammy Jellyfish) de 64 bits, sin entorno gráfico. Se usa Ubuntu 22.04 por ser LTS con soporte hasta 2027.

Servidor de dominio: Windows Server 2016 (Active Directory, DNS, DHCP).

Servidores web (Backends)

- Apache 2 con mod_rewrite habilitado y VirtualHost configurado.
- PHP con módulos libapache2-mod-php y php-mysql.
- Cliente NFS (nfs-common) para montar el directorio compartido.
- Cliente MySQL para verificar la conectividad con el SGBD.

Servidor NFS

- Nfs-kernel-server para exportar el directorio /srv/app.

Balanceador de carga

- Nginx configurado como proxy inverso con bloque upstream ip_hash.

Motor de base de datos

- MySQL Server 8 con charset utf8mb4 y política de iptables DROP por defecto.

Herramientas de gestión y automatización

- Vagrant: Automatiza la creación, configuración y aprovisionamiento de toda la infraestructura mediante scripts shell.
- VirtualBox: Hypervisor para ejecutar las máquinas virtuales. Compatible con Windows, macOS y Linux.
- Python 3 + Tkinter: Herramienta de monitorización y administración con interfaz gráfica.

4. Diseño de red

El diseño de red está segmentado en cuatro redes privadas para proporcionar mayor seguridad y aislamiento. Esta arquitectura previene el acceso directo del balanceador a la base de datos, obligando a que toda comunicación pase por los backends.

Topología de red

La infraestructura utiliza una topología en multicapa con cuatro redes privadas virtuales separadas:

- LAN1 (10.0.60.0/23): red de servicios. Aloja el Windows Server y el servidor FTP.
- LAN2 (10.0.50.0/23): red de acceso exterior. Conecta el router con el balanceador.
- LAN Interna (10.0.30.0/23): red de backends y NFS. Comunicación interna de la capa web.
- LAN Datos (10.0.20.0/23): red exclusiva para la base de datos. Solo accesible desde los backends.

Los backends disponen de dos interfaces de red, actuando como puente entre la LAN Interna y la LAN Datos. El balanceador y el servidor de base de datos están conectados únicamente a una red.

Direccionamiento IP

Servidor	IP principal	IP secundaria	Red	Hostname
SGBD	10.0.20.10	—	LAN Datos	sgbd
Backend 1	10.0.30.10	10.0.20.20	LAN Interna	backend1
Backend 2	10.0.30.20	10.0.20.30	LAN Interna	backend2
NFS	10.0.30.40	—	LAN Interna	nfs
Balanceador	10.0.50.10	10.0.30.30	LAN2	balanceador
Router/NAT	10.0.50.40	10.0.60.40	LAN2/LAN1	router
Windows Server	10.0.60.10	—	LAN1	windows
FTP	10.0.60.20	—	LAN1	ftp

Puertos y protocolos utilizados

Servicio	Puerto	Protocolo	Servidor	Red
HTTP (público)	80	TCP/HTTP	Balanceador	LAN2
HTTP (backend)	80	TCP/HTTP	Backend 1 y 2	LAN Interna
MySQL	3306	TCP/MySQL	SGBD	LAN Datos
NFS	2049	TCP/NFS	NFS	LAN Interna
FTP (control)	21	TCP/FTP	FTP	LAN1

Servicio	Puerto	Protocolo	Servidor	Red
FTP (pasivo)	40000-50000	TCP	FTP	LAN1
DNS	53	UDP/TCP	Windows Server	LAN1
SSH	22	TCP/SSH	Todos	Vagrant (host)

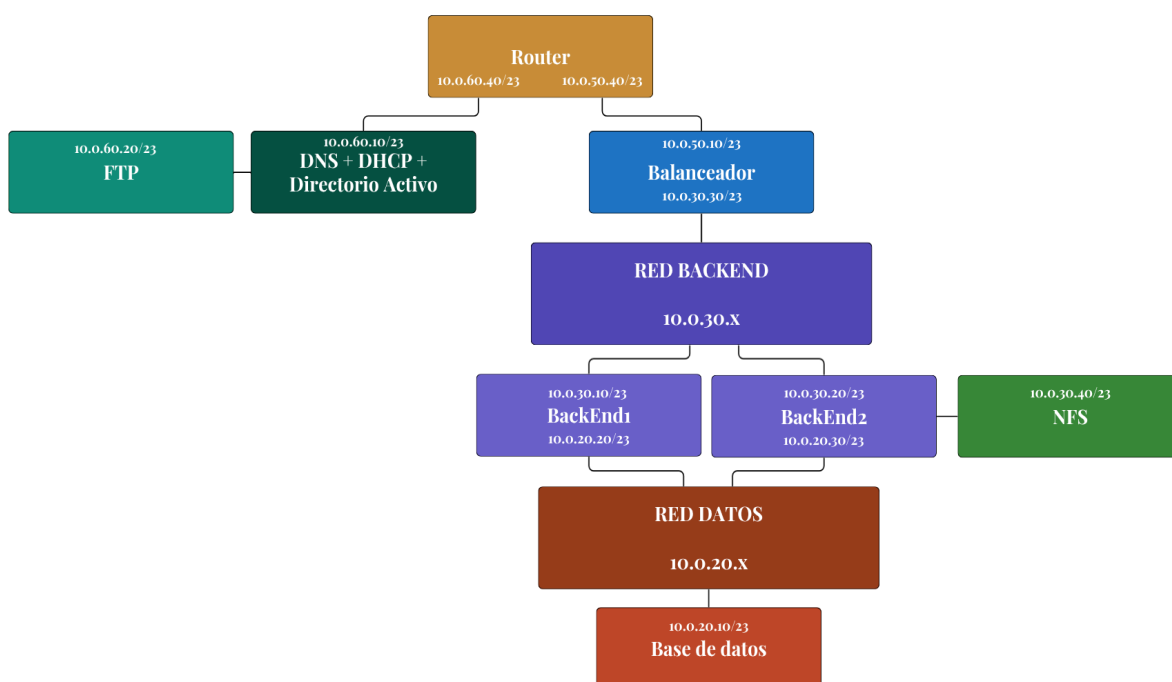
Esquema de seguridad perimetral

La seguridad perimetral se implementa separando cada capa para que no tengan acceso directo entre sí.

Principio de defensa en profundidad

- Capa 1 — Acceso externo: el tráfico entra por el balanceador Nginx en 10.0.50.10. Solo el puerto 80 está expuesto desde la red de la clínica.
- Capa 2 — Balanceador: Nginx actúa como punto único de entrada, balancea la carga con ip_hash y está completamente aislado de la capa de datos.
- Capa 3 — Backends: los servidores Apache+PHP son los únicos componentes con acceso a ambas redes (LAN Interna y LAN Datos). Actúan como proxy hacia la base de datos.
- Capa 4 — Base de datos: MySQL está completamente aislado en la LAN Datos. Política iptables DROP por defecto; solo acepta conexiones en el puerto 3306 desde 10.0.20.20 y 10.0.20.30.
- Capa 5 — Servicios de red: el Windows Server y el FTP están en la LAN1, separada del resto de la infraestructura web.

Diagrama de Red



5. Servidores web (Backends)

Función dentro de la infraestructura

Los backends constituyen la capa de aplicación de la infraestructura, actuando como intermediarios entre el balanceador y la base de datos.

- Procesamiento de peticiones HTTP: Reciben tráfico del balanceador y ejecutan el código PHP de la aplicación clínica.
- Intermediación segura: Son el único componente con acceso tanto a la LAN Interna (balanceador) como a la LAN Datos (base de datos).
- Alta disponibilidad: Con dos servidores idénticos el sistema mantiene operatividad si uno falla.
- Código compartido vía NFS: Ambos backends montan /srv/app desde el servidor NFS, garantizando coherencia sin sincronización manual.

Configuración básica

Configuración de Apache

VirtualHost configurado en /etc/apache2/sites-available/clinica.conf:

```
<VirtualHost *:80>
  ServerName ${HOSTNAME_VM}
  DocumentRoot /var/www/html
  <Directory /var/www/html>
    AllowOverride All
    Require all granted
    DirectoryIndex index.php index.html
  </Directory>
  ErrorLog ${APACHE_LOG_DIR}/clinica_error.log
</VirtualHost>
```

Montaje NFS

Entrada en /etc/fstab para montar el código compartido en el arranque:

```
10.0.30.40:/srv/app /var/www/html nfs defaults,_netdev 0 0
```

Virtual hosts / sitios web

Ambos backends sirven la aplicación Clínica General desde /var/www/html, montado desde el servidor NFS. La aplicación se conecta a la base de datos MySQL en 10.0.20.10 mediante PDO con usuario clinica_user.

6. Balanceador de carga

Función del balanceador

Nginx actúa como punto de entrada único para todo el tráfico HTTP externo. Sus funciones principales son:

- Distribución de tráfico entre los backends con ip_hash.
- Persistencia de sesión: El ip_hash garantiza que un mismo cliente siempre llegue al mismo backend, lo que es necesario para las sesiones PHP.
- Primera barrera de seguridad: Aislado de la LAN Datos, no puede alcanzar la base de datos directamente.
- Enrutamiento hacia LAN1: Ruta estática hacia 10.0.60.0/23 (Windows Server y FTP) configurada vía netplan.

Tipo de balanceo

Algoritmo implementado: ip_hash (no round robin).

Este algoritmo deriva un hash de la IP de origen del cliente y siempre lo enruta al mismo backend. Esto garantiza la persistencia de las sesiones PHP sin necesidad de almacenamiento de sesiones compartido.

Ventajas del ip_hash:

- Persistencia de sesión sin configuración adicional.
- Distribución determinista por cliente.
- Compatible con aplicaciones que almacenan estado en sesión de servidor.

Configuración del bloque upstream

```
upstream backends {
    ip_hash;
    server 10.0.30.10;
    server 10.0.30.20;
}

server {
    listen 80;
    server_name clinicageneral.local;
    location / {
        proxy_pass      http://backends;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

7. Base de datos

Tipo de base de datos

Motor: MySQL Server 8.

Tipo: base de datos relacional con charset utf8mb4 y collation utf8mb4_unicode_ci.

Arquitectura

Arquitectura actual: base de datos única en servidor dedicado (10.0.20.10) aislado en la LAN Datos. Solo los backends en 10.0.20.20 y 10.0.20.30 tienen acceso permitido.

Tablas del esquema clínico

Tabla	Descripción
DEPARTAMENTO	Departamentos de la clínica (8 departamentos)
ESPECIALIDAD	Especialidades médicas disponibles
GRUPO	Grupos de trabajo multidisciplinares
PACIENTE	Datos personales de los pacientes
TIPO_PRUEBA	Catálogo de análisis de laboratorio
EMPLEADO	Personal con rol, departamento y especialidad
EMPLEADOS_GRUPOS	Relación N:M entre empleados y grupos
MEDICO_PACIENTE	Asignación de pacientes a médicos
HISTORIAL_MEDICO	Historial clínico por paciente (1:1)
HISTORIAL_CLINICO	Entradas de historial con tipo de consulta (ENUM)
CITA	Citas entre pacientes y médicos con estado (pendiente/programada/completada/cancelada)
CONSULTA	Consultas médicas con diagnóstico y tratamiento
RECETA	Recetas emitidas en consultas
DETALLE_RECETA	Medicamentos, dosis y duración por receta
SOLICITUD_ANALISIS	Solicitudes de análisis generadas en consulta
RESULTADO_LABORATORIO	Resultados numéricos de los análisis
FACTURA	Facturas asociadas a pacientes y consultas
USUARIO	Cuentas de acceso con rol y contraseña SHA2(256)
LOG_ACCESO	Registro de inicios de sesión con IP real y timestamp

Usuarios y permisos

Usuario	Contraseña	Host	Permisos
clinica_user	1234	10.0.20.%, 10.0.30.%	Todos en base de datos clinica
clinica_admin	Admin_Cl1nica!	localhost	ALL PRIVILEGES

El usuario clinica_user solo puede conectarse desde las IPs de los backends y tiene acceso únicamente a la base de datos clinica.

Conexión segura con el servidor web

Método de conexión:

- Protocolo: TCP/IP (puerto 3306).
- Red utilizada: LAN Datos (10.0.20.0/23), completamente aislada del exterior.
- Autenticación: usuario/contraseña con contraseñas PHP hasheadas con SHA2(password, 256) en MySQL.

Parámetros de conexión desde PHP (config/db.php):

```
$host = '10.0.20.10';
$dbname = 'clinica';
$usuario = 'clinica_user';
$password = '1234';
$pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4",
    $usuario, $password);
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

Medidas de seguridad implementadas:

- Restricción de IP vía iptables: MySQL solo acepta conexiones desde 10.0.20.20 y 10.0.20.30.
- Política DROP en iptables: todo el tráfico entrante al SGBD se rechaza salvo las reglas explícitas.
- Contraseñas con SHA2: ninguna contraseña se almacena en texto plano.

8. Dominio Windows Server

El dominio clinicageneral.local está implementado en Windows Server con Active Directory, DNS y DHCP en 10.0.60.10. La estructura organizativa refleja los departamentos reales de la clínica.

Estructura de unidades organizativas

Unidad organizativa	Usuarios de ejemplo
Dirección	director.general, subdirector
Administración	jefe.administracion, facturacion1
RRHH	jefe.rrhh, rrhh.tecnico1
Informática	admin.sistema, tecnico.it1, tecnico.it2
Especialistas	esp.cardiologia, esp.dermatologia, esp.traumatologia
Enfermería	enf.ana, enf.carlos
Recepción	recepcion1, recepcion2
Laboratorio	lab.tecnico1, lab.tecnico2

Grupos de seguridad

Grupo	Ámbito	Descripción
DIR_Usuarios	Global	Dirección con acceso a recursos confidenciales
ADM_Usuarios	Global	Administración y Facturación
RRHH_Usuarios	Global	Recursos Humanos y gestión de personal
IT_Usuarios	Global	Departamento IT con acceso a herramientas internas
IT_Admins	Global	Administradores de dominio con permisos elevados
ESP_Usuarios	Global	Especialistas con acceso a historial y agenda
ENF_Usuarios	Global	Enfermería con acceso a historial y seguimiento
REC_Usuarios	Global	Recepción con acceso a gestión de citas
LAB_Usuarios	Global	Laboratorio con acceso a resultados y análisis

Servicios DNS y DHCP

DNS: zona directa e inversa para clinicageneral.local. Los servidores Linux apuntan a 10.0.60.10 como DNS primario. El servidor FTP tiene el fichero /etc/resolv.conf bloqueado con chatr +i para que no se sobrescriba.

DHCP: reservas para todos los equipos fijos de la clínica, con rangos dinámicos para dispositivos de usuario.

9. Router / NAT — Interconexión entre redes

El router es la máquina virtual encargada de interconectar la red LAN2 (donde reside el balanceador) con la red LAN1 (donde se encuentran el Windows Server y el servidor FTP). Sin este componente, el balanceador no tendría visibilidad sobre los servicios de dominio de Active Directory.

Función dentro de la infraestructura

- Puerta de enlace entre LAN2 (10.0.50.0/23) y LAN1 (10.0.60.0/23): el router tiene una interfaz en cada red y reenvía el tráfico entre ambas.
- Permite que el balanceador Nginx alcance el Windows Server (10.0.60.10): la ruta estática hacia 10.0.60.0/23 configurada en el balanceador vía netplan apunta al router como gateway (10.0.50.40).
- Actúa como NAT: Puede enmascarar el tráfico saliente de la LAN Interna hacia el exterior.

Direccionamiento

Interfaz	IP	Red	Conecta con
eth1 (LAN2)	10.0.50.40	10.0.50.0/23	Balanceador (10.0.50.10)
eth2 (LAN1)	10.0.60.40	10.0.60.0/23	Windows Server (10.0.60.10), FTP (10.0.60.20)

Ruta estática en el balanceador

Para que el balanceador pueda llegar a la red LAN1, se configura una ruta estática en su netplan apuntando al router:

```
# /etc/netplan/01-netcfg.yaml (en el balanceador)
routes:
  - to: 10.0.60.0/23
    via: 10.0.50.40 # IP del router en LAN2
```

Esta ruta permite que el balanceador resuelva nombres DNS contra el Windows Server (10.0.60.10) y que los clientes de dominio puedan autenticarse a través de Active Directory.

IP Forwarding en el router

El reenvío de paquetes entre interfaces está activado en el kernel del router:

```
# Activar ip_forward en /etc/sysctl.conf
net.ipv4.ip_forward = 1

# Aplicar sin reiniciar
sudo sysctl -p
```

Sin este parámetro, el router recibiría los paquetes pero no los reenviaría entre sus interfaces de red.

Flujo de comunicación: Balanceador a Windows Server

El flujo de una petición de resolución DNS desde el balanceador hacia el Windows Server sigue este camino:

- 1. El balanceador (10.0.50.10) envía una consulta DNS a 10.0.60.10.
- 2. La tabla de rutas del balanceador indica que 10.0.60.0/23 es accesible via 10.0.50.40 (router).
- 3. El router recibe el paquete en eth1 (LAN2) y lo reenvía por eth2 (LAN1) hacia 10.0.60.10.
- 4. El Windows Server responde. El router reenvía la respuesta de vuelta al balanceador.

10. Sistema de respaldo (Backups)

Objetivos del sistema de copias de seguridad

El sistema de copias de seguridad tiene como objetivo garantizar la continuidad del servicio y la recuperación ante desastres mediante la preservación de:

- Integridad de datos: Protección de la información almacenada en la base de datos clínica (pacientes, historiales, citas).
- Disponibilidad: Capacidad de restaurar el servicio en tiempo mínimo ante cualquier incidente.

Elementos respaldados

Base de datos

Contenido: base de datos clínica completa con las 19 tablas, índices, datos de pacientes, citas e historiales.

Método: mysqldump con compresión gzip.

```
mysqldump -u clinica_admin -pAdmin_Cl1nica! clinica | \
gzip > /backup/clinica_$(date +%Y%m%d_%H%M%S).sql.gz
```

Código de la aplicación (NFS)

Contenido: directorio /srv/app con todos los archivos PHP, CSS, JS y configuración.

Método: tar con compresión.

```
tar -czf /backup/app_$(date +%Y%m%d).tar.gz /srv/app/
```

Configuraciones de los servidores

Archivos críticos:

- /etc/nginx/sites-available/clinica (balanceador)
- /etc/apache2/sites-available/clinica.conf (backends)
- /etc/exports (servidor NFS)
- /etc/mysql/mysql.conf.d/mysqld.cnf (SGBD)
- /etc/haproxy/haproxy.cfg (si se usa)

```
#!/bin/bash
BDIR="/backup/configs_$(date +%Y%m%d)"
mkdir -p $BDIR
cp /etc/nginx/sites-available/clinica $BDIR/
cp /etc/apache2/sites-available/clinica.conf $BDIR/
cp /etc/exports $BDIR/
tar -czf $BDIR.tar.gz $BDIR/ && rm -rf $BDIR
```

Procedimiento de restauración

Restauración de base de datos

```
# 1. Detener backends
vagrant ssh backend1 -c "sudo systemctl stop apache2"
vagrant ssh backend2 -c "sudo systemctl stop apache2"

# 2. Restaurar dump
vagrant ssh sgbd
gunzip < /backup/clinica_20260529.sql.gz | mysql -u clinica_admin -pAdmin_Cl1nica! clinica

# 3. Verificar restauración
mysql -u clinica_admin -pAdmin_Cl1nica! clinica -e "SELECT COUNT(*) FROM PACIENTE;"

# 4. Reiniciar backends
vagrant ssh backend1 -c "sudo systemctl start apache2"
vagrant ssh backend2 -c "sudo systemctl start apache2"
```

Restauración del código (NFS)

```
# 1. Extraer backup
tar -xzf /backup/app_20260529.tar.gz -C /tmp/

# 2. Copiar archivos
sudo cp -r /tmp/srv/app/* /srv/app/

# 3. Ajustar permisos
sudo chown -R nobody:nogroup /srv/app/
sudo chmod -R 755 /srv/app/
```

11. Seguridad

Gestión de usuarios y accesos

Principio de mínimo privilegio

Cada usuario y servicio tiene únicamente los permisos necesarios para realizar su función:

Usuario	Servicio	Permisos	Restricciones
clinica_user	MySQL	Todo en BD clinica	Solo desde 10.0.20.20 y 10.0.20.30
clinica_admin	MySQL	ALL PRIVILEGES	Solo localhost en SGBD
www-data	Apache	Lectura /var/www/html	Usuario de sistema
nobody:nogroup	NFS	Propietario de /srv/app	Sin shell de login
admin	Aplicación web	Panel de administración	Contraseña SHA2(256)

Políticas de contraseñas

Requerimientos para producción:

- Longitud mínima: 12 caracteres (mínimo 6 en la aplicación, recomendado 12 en producción).
- Caracteres complejos: mayúsculas, minúsculas, números y símbolos especiales.
- Almacenamiento: SHA2(password, 256) ejecutado en MySQL, nunca en texto plano.
- Rotación recomendada: cada 90 días para cuentas de administración.

Seguridad iptables (SGBD)

Regla	Descripción
Política INPUT DROP	Todo el tráfico entrante al SGBD se rechaza por defecto
Política FORWARD DROP	Todo el tráfico de reenvío se rechaza por defecto
Puerto 3306	Solo accesible desde 10.0.20.20 y 10.0.20.30 (backends)
Puerto 22 (SSH)	Solo desde 10.0.0.0/8 (red interna)
LOG IPTABLES-DROP	Paquetes rechazados registrados con prefijo IPTABLES-DROP:

Actualizaciones y parches

```
# Actualizar todos los paquetes (en cada servidor)
sudo apt-get update && sudo apt-get upgrade -y
```

```
# Verificar actualizaciones disponibles
sudo apt list --upgradable
```

Protección frente a ataques comunes

Tipo de ataque	Mitigación implementada	Mejora recomendada
Acceso no autorizado a BD	iptables DROP + restricción por IP	Certificados de cliente MySQL
Inyección SQL	PDO con prepared statements en toda la aplicación	Auditoría periódica de queries
XSS	htmlspecialchars() en todas las salidas de la app	Content Security Policy (CSP)
Fuerza bruta al login	Contraseñas SHA2 + log de intentos en LOG_ACCESO	Fail2ban, MFA
Sniffing de red	Redes segmentadas y aisladas por función	SSL/TLS en producción
DDoS	Balanceo de carga distribuye la presión	Rate limiting en Nginx, Cloudflare

Buenas prácticas de seguridad

- Segmentación de red: Mantener el aislamiento entre capas y no permitir acceso directo del balanceador a la base de datos.
- Logging y auditoría: La tabla LOG_ACCESO registra todos los inicios de sesión con IP real (HTTP_X_FORWARDED_FOR), rol y timestamp.
- Actualizaciones regulares: Mantener todos los servidores actualizados con los últimos parches de seguridad.
- Deshabilitación de servicios innecesarios: Reducir la superficie de ataque eliminando servicios no utilizados.
- Firewall iptables: Política DROP por defecto en el SGBD con reglas explícitas solo para los servicios necesarios.

12. Monitorización y mantenimiento (Script Python)

Se ha desarrollado una consola de administración centralizada en Python con interfaz gráfica Tkinter. Permite supervisar el estado del hardware, gestionar el ciclo de vida de las VMs y auditar la seguridad desde una sola herramienta.

Herramientas de monitorización

Logs del sistema:

- `/var/log/apache2/clinica_access.log` — peticiones HTTP en los backends.
- `/var/log/apache2/clinica_error.log` — errores de Apache.
- `/var/log/nginx/access.log` — tráfico en el balanceador.
- `/var/log/nginx/error.log` — errores de Nginx.
- `/var/log/mysql/error.log` — errores de MySQL.
- `/var/log/auth.log` — intentos de autenticación SSH.
- `journalctl` — logs del sistema (systemd).
- Tabla LOG_ACCESO — registro de logins en la aplicación web con IP y timestamp.

Parámetros supervisados

Categoría	Métrica	Umbral de advertencia	Umbral crítico
CPU	Uso de CPU (%)	> 70%	> 90%
RAM	Memoria utilizada (%)	> 80%	> 95%
Disco	Espacio libre	< 20%	< 10%
Red	Tráfico (Mbps)	> 80% capacidad	> 95% capacidad
Servicios	Estado (UP/DOWN)	DOWN 1 servicio	DOWN servicio crítico
Seguridad	Logins fallidos	> 10 en 1 hora	> 50 en 1 hora

Scripts de monitorización

Script	Función
<code>cpu.sh</code>	Uptime, carga media y top de procesos por consumo de CPU
<code>servicios_web.sh</code>	Estado de apache2, nginx y mysql (systemctl status)
<code>vagrant_control.sh</code>	Orquestador de comandos Vagrant para las VMs del proyecto
<code>discos_vdi.sh</code>	Espacio ocupado por discos virtuales .vdi y .vmdk
<code>login_fallidos.sh</code>	Análisis de <code>/var/log/auth.log</code> para detectar intentos fallidos
<code>trafico_red.sh</code>	Interfaces de red, rutas y conexiones TCP activas

Comandos de monitorización manual

```
# CPU y memoria
top / htop

# Uso de disco
df -h && du -sh /srv/app/*

# Estado de servicios
systemctl status apache2 nginx mysql nfs-kernel-server

# Conexiones de red
ss -tuln

# Logs en tiempo real
tail -f /var/log/apache2/clinica_error.log
journalctl -u apache2 -f

# Verificar NFS
showmount -e 10.0.30.40
mount | grep nfs

# Verificar MySQL
mysql -h 10.0.20.10 -u clinica_user -p1234 clinica -e "SHOW TABLES;"
```


13. Procedimientos operativos

Arranque y parada de servicios

Orden de arranque correcto

1. Base de datos MySQL (primero, los backends dependen de ella):

```
agrant up sgb  
agrant ssh sgb -c "sudo systemctl status mysql"
```

2. Servidor NFS (antes que los backends, que montan desde él):

```
agrant up nfs  
agrant ssh nfs -c "sudo systemctl status nfs-kernel-server"
```

3. Servidores backend (Apache + PHP):

```
agrant up backend1 backend2  
agrant ssh backend1 -c "sudo systemctl status apache2"  
agrant ssh backend2 -c "sudo systemctl status apache2"
```

4. Balanceador Nginx:

```
agrant up balanceador  
agrant ssh balanceador -c "sudo systemctl status nginx"
```

5. Router y servicios LAN1:

```
agrant up router ftp
```

6. Orden de parada correcto

```
agrant halt balanceador  
agrant halt backend1 backend2  
agrant halt nfs  
agrant halt sgb  
agrant halt router ftp
```

```
# Parada completa del entorno  
agrant halt
```

Alta/baja de un servidor backend

Dar de alta un nuevo backend

```
# 1. Añadir en Vagrantfile  
config.vm.define "backend3" do |b|  
  b.vm.hostname = "backend3"  
  b.vm.network "private_network", ip: "10.0.30.50"  
  b.vm.network "private_network", ip: "10.0.20.40"  
  b.vm.provision "shell", path: "provision/backend.sh"  
end  
  
# 2. Levantar el nuevo backend  
agrant up backend3  
  
# 3. Añadir al bloque upstream en el balanceador
```

```
vagrant ssh balanceador  
sudo nano /etc/nginx/sites-available/clinica  
# Añadir: server 10.0.30.50; dentro del bloque upstream  
sudo nginx -t && sudo systemctl reload nginx
```

Dar de baja un backend

```
# 1. Eliminar del upstream en el balanceador  
vagrant ssh balanceador  
sudo nano /etc/nginx/sites-available/clinica  
sudo systemctl reload nginx  
  
# 2. Detener el servidor  
vagrant halt backend2
```

Actualización del sistema

```
# 1. Actualizar backend2 (backend1 sigue atendiendo)  
vagrant ssh backend2  
sudo apt-get update && sudo apt-get upgrade -y  
sudo systemctl restart apache2  
curl http://10.0.30.20  
  
# 2. Actualizar backend1  
vagrant ssh backend1  
sudo apt-get update && sudo apt-get upgrade -y  
sudo systemctl restart apache2  
  
# 3. Actualizar SGBD (con precaución)  
vagrant ssh sgbd  
sudo apt-get update && sudo apt-get upgrade -y  
  
# 4. Actualizar balanceador  
vagrant ssh balanceador  
sudo apt-get update && sudo apt-get upgrade -y  
sudo systemctl restart nginx
```

14. Resolución de problemas

Problemas habituales

Síntoma	Causa probable	Verificación
No se puede acceder a la app	Balanceador caído o NFS no montado	<code>vagrant status balanceador; mount grep nfs</code>
Error 502 Bad Gateway	Backends caídos o Apache no arrancado	<code>systemctl status apache2 en backend1/2</code>
Error de conexión a BD	MySQL caído o iptables bloqueando	<code>systemctl status mysql; iptables -L -n</code>
Página en blanco (PHP)	Error PHP no visible	<code>tail /var/log/apache2/clinica_error.log</code>
NFS no monta al arrancar	sgbd arrancó antes que nfs	<code>mount -a en los backends tras levantar nfs</code>
Sesión no persiste	ip_hash cambia de backend	Verificar ip_hash en upstream nginx
Login incorrecto en la app	Contraseña no coincide con SHA2	Verificar hash en tabla USUARIO

Diagnóstico básico

Metodología de resolución de problemas

```
# 1. Estado general de todas las VMs
vagrant status

# 2. Probar conectividad básica
curl http://10.0.50.10

# 3. Verificar servicios
vagrant ssh balanceador -c "sudo systemctl status nginx"
vagrant ssh backend1 -c "sudo systemctl status apache2"
vagrant ssh sgbd -c "sudo systemctl status mysql"
vagrant ssh nfs -c "sudo systemctl status nfs-kernel-server"

# 4. Revisar logs más recientes
vagrant ssh backend1 -c "sudo tail -50 /var/log/apache2/clinica_error.log"
vagrant ssh sgbd -c "sudo tail -50 /var/log/mysql/error.log"

# 5. Verificar conectividad de red
vagrant ssh backend1 -c "ping -c 3 10.0.20.10" # backend1 -> SGBD
vagrant ssh balanceador -c "ping -c 3 10.0.30.10" # balanceador -> backend1

# 6. Verificar puertos escuchando
vagrant ssh balanceador -c "sudo ss -tuln | grep 80"
vagrant ssh sgbd -c "sudo ss -tuln | grep 3306"
```

Comprobaciones recomendadas

Checklist de verificación de los servidores

- ¿Están todas las VMs corriendo? (vagrant status)
- ¿Responden los servicios? (systemctl status)
- ¿Está el NFS montado en los backends? (mount | grep nfs)
- ¿Hay conectividad entre servidores? (ping entre capas)
- ¿Están los puertos abiertos? (ss -tuln)
- ¿Hay espacio en el disco? (df -h)
- ¿Hay recursos disponibles? (top / htop)
- ¿Hay errores en los logs? (tail -f)
- ¿Funciona la app desde línea de comandos? (curl)

Logs relevantes

Componente	Ubicación del log	Comando
Nginx (balanceador)	/var/log/nginx/access.log	vagrant ssh balanceador -c "tail -f /var/log/nginx/access.log"
Apache (backend1)	/var/log/apache2/clinica_access.log	vagrant ssh backend1 -c "tail -f /var/log/apache2/clinica_access.log"
Apache (error)	/var/log/apache2/clinica_error.log	vagrant ssh backend1 -c "tail -f /var/log/apache2/clinica_error.log"
MySQL (error)	/var/log/mysql/error.log	vagrant ssh sgbd -c "tail -f /var/log/mysql/error.log"
Auth SSH	/var/log/auth.log	vagrant ssh sgbd -c "tail -f /var/log/auth.log"
Sistema (todos)	journalctl	vagrant ssh backend1 -c "journalctl -xe"

15. Aplicación web — /app (práctica Marzo–Junio)

Implantación de Aplicaciones Web, desarrollado durante el período de marzo a junio.

La carpeta app/ contiene la aplicación web de Clínica General, desarrollada en PHP puro sin framework siguiendo el patrón Modelo Vista Controlador (MVC). Permite gestionar citas, pacientes, historiales clínicos y empleados a través de cuatro roles de acceso diferenciados:

- Paciente: solicita y cancela citas, consulta su historial clínico y gestiona su perfil.
- Médico: gestiona sus citas y pacientes asignados, registra entradas en el historial y dispone de un calendario FullCalendar.
- Recepcionista: administra todas las citas, asigna médicos a citas pendientes y da de alta pacientes.
- Administrador: gestiona empleados, consulta estadísticas globales y revisa el log de accesos.

Controladores

Nombre	Archivo	Descripción
Autenticación	AuthController.php	Login y logout. Registra IP real en LOG_ACCESO.
Registro	RegisterController.php	Alta de paciente con transacción atómica PACIENTE + USUARIO.
Administrador	AdminController.php	CRUD de empleados, toggle activo y reset de contraseña.
Citas (médico)	CitaController.php	Guardar, actualizar, eliminar y completar citas.
Recepcionista	RecepcionistaController.php	CRUD de citas y asignación de médico (pendiente→programada).
Pacientes	PacienteController.php	Endpoint JSON de búsqueda, asignar/desasignar y guardar historial.
Perfil paciente	PerfilPacienteController.php	Editar perfil, cambiar contraseña, solicitar y cancelar citas.
Configuración médico	ConfiguracionController.php	Actualiza datos personales (refresca \$_SESSION) y contraseña.
Calendario	CalendarioController.php	API REST JSON para FullCalendar. HTTP 403 sin sesión.

Modelos

Nombre	Archivo	Descripción
Usuario	UsuarioModel.php	buscarPorCredenciales con LEFT JOIN, COALESCE y SHA2.
Administrador	AdminModel.php	Estadísticas, ranking de citas (SUM condicional), CRUD empleados.
Cita	CitaModel.php	Modelo más extenso. Queries distintas por rol.
Paciente	PacienteModel.php	Historial clínico, búsqueda por DNI/teléfono y gestión de perfil.
Configuración	ConfiguracionModel.php	Datos del médico con JOIN a USUARIO. Verificación SHA2.

Vistas por rol

Rol	Vista principal	Características
Administrador	panel_admin.php	6 estadísticas, top 5 médicos con barras proporcionales, log de accesos.
Administrador	empleados_admin.php	Tabla paginada (15/pág), 3 modales Bootstrap con json_encode+JS.
Médico	panel_medico.php	FullCalendar con eventos JSON, estadísticas y próximas citas.
Médico	pacientes_medico.php	Búsqueda AJAX con fetch() + Enter. Historial pre-cargado.
Paciente	citas_paciente.php	Citas en 4 estados con array_filter + arrow functions PHP 7.4+.
Recepcionista	panel_recepcionista.php	Cola de pendientes con filtrado por especialidad vía atributo hidden.

Tecnologías usadas en /app

Tecnología	Uso
PHP 8 (sin framework)	MVC artesanal con require_once. Control de acceso por rol en \$_SESSION.
PDO + MySQL	Prepared statements, SHA2, transacciones, INSERT IGNORE, COALESCE.
Bootstrap 5.3.3 + Bootstrap Icons	CDN jsDelivr. Grid, modales, badges, navbar e iconografía.
Google Fonts	Playfair Display (títulos) + DM Sans (cuerpo).
CSS Custom Properties	Variables --verde, --crema, --borde en :root.

Tecnología	Uso
FullCalendar	CDN. Dashboard del médico. Consume CalendarioController como API REST.
JavaScript (ES6)	fetch() para AJAX, template literals, bootstrap.Modal programático.
json_encode PHP→JS	Datos pre-cargados inyectados como constantes JS.

16. Documentación adicional

Credenciales de prueba

Aplicación web

Usuario	Contraseña	Rol
admin	admin1234	Administrador
pedro.alonso	medico1234	Médico
carmen.torres	enf1234	Enfermera
antonio.ruiz	rec1234	Recepcionista
miguel.serrano	lab1234	Laboratorio
maria.garcia	pac1234	Paciente

Las contraseñas se almacenan como SHA2(password, 256) en la tabla USUARIO.

Servidor FTP

Todos los usuarios FTP: contraseña Clinica2025!

MySQL

Usuario	Contraseña	Acceso
clinica_user	1234	Desde backends (10.0.20.%, 10.0.30.%)
clinica_admin	Admin_Cl1nica!	Localhost en SGBD

Tecnologías usadas para elaborar el proyecto y documento

Virtualización e infraestructura

Vagrant: aprovisionamiento automatizado de las máquinas virtuales mediante scripts shell.

VirtualBox: virtualización de las máquinas virtuales en el sistema host.

Ubuntu Server 22.04 LTS: sistema operativo de todas las VMs Linux.

Windows Server: sistema operativo del servidor de dominio.

Tecnologías empleadas

Nginx: balanceador de carga con ip_hash.

Apache 2: servidor web en los backends.

PHP 8: lenguaje de programación del backend de la aplicación web.

MySQL 8: sistema de gestión de base de datos relacional.

NFS: sistema de ficheros en red para código compartido.

vsftpd: servidor FTP para intercambio de archivos entre departamentos.

Python 3 + Tkinter: herramienta de monitorización y administración.

Active Directory: gestión centralizada de usuarios y equipos del dominio.

Lenguajes

PHP 8: lógica de la aplicación web (MVC, PDO, sesiones).

HTML5 + CSS3 + Bootstrap 5.3.3: estructura y diseño de la interfaz web.

JavaScript ES6: interactividad, AJAX con fetch() y FullCalendar.

Bash: aprovisionamiento automático de todas las VMs con Vagrant.

Python 3: herramienta de monitorización con interfaz gráfica Tkinter.

Diseño del documento

Google Docs: redacción y maquetación del documento.

Canva: diagramas de red de la infraestructura.